```
template<typename T>
class lock_free_queue
{
private:
    static void free_external_counter(counted_node_ptr &old_node_ptr)
    {
        node* const ptr=old_node_ptr.ptr;
        int const count_increase=old_node_ptr.external_count-2;

        node_counter old_counter=
            ptr->count.load(std::memory_order_relaxed);
        node_counter new_counter;
        do
        {
            new_counter=old_counter;                          ❶
            --new_counter.external_counters;        <-┘
            new_counter.internal_count+=count_increase;     <-❷
        }
        while(!ptr->count.compare_exchange_strong(     <-❸
              old_counter,new_counter,
              std::memory_order_acquire,std::memory_order_relaxed));

        if(!new_counter.internal_count &&
           !new_counter.external_counters)
        {
            delete ptr;     <-❹
        }
    }
};
```

The counterpart to `increase_external_count()` is `free_external_counter()`. This is similar to the equivalent code from `lock_free_stack::pop()` in listing 7.11 but modified to handle the `external_counters` count. It updates the two counts using a single `compare_exchange_strong()` on the whole `count` structure ❸, just as you did when decreasing the `internal_count` in `release_ref()`. The `internal_count` value is updated as in listing 7.11 ❷, and the `external_counters` value is decreased by one ❶. If *both* the values are now zero, there are no more references to the node, so it can be safely deleted ❹. This has to be done as a single action (which therefore requires the compare/exchange loop) to avoid a race condition. If they're updated separately, two threads may both think they are the last one and thus both delete the node, resulting in undefined behavior.